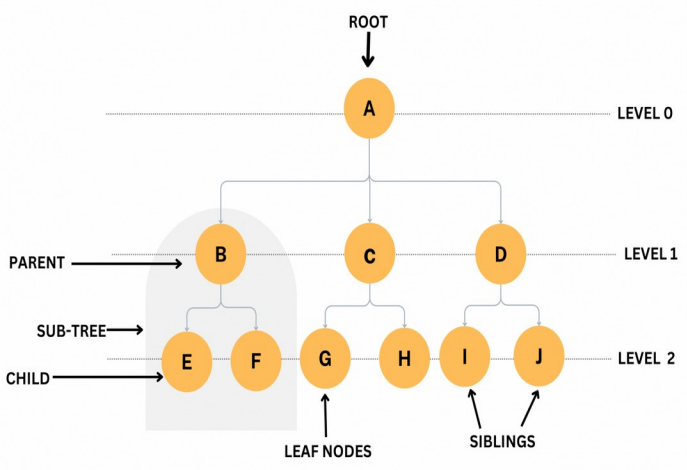
Tree Data Structure:

A tree is a non-linear data structure consisting of nodes connected by edges. It is used to represent hierarchical structures, such as file systems, computer directories, and organization charts. Trees are a fundamental data structure used in computer science, and they have many applications, such as in search algorithms, decision-making processes, and data compression.



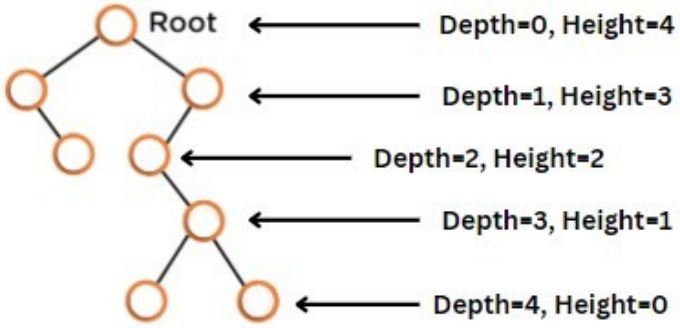
The topmost node in a tree is called the root node, which has no parent nodes. Each node in a tree can have one or more child nodes, except for the leaf nodes, which have no children. Nodes that share the same parent node are called siblings.

1. The tree is a connected graph: A tree is a connected graph, which means that there is a path from any node to any other node in the tree.
2. The tree has a unique root node: A tree has a unique root node, which is the topmost node in the tree.
3. Each child node has its own subtree: Each node in a tree has its own subtree, which is a subtree that includes all the nodes that are descendants of the node.
4. The tree is acyclic: A tree is acyclic, which means that there are no cycles in the tree.
5. The tree may be empty: A tree may be empty, which means that it has no nodes.

The terminology used in trees includes the following:

1. **Root:** The topmost node in a tree, which has no parent nodes.
2. **Parent:** A node that has one or more child nodes.
3. **Child:** A node that has a parent node.
4. **Siblings:** Nodes that have the same parent node.

5. **Leaf:** A node with no children.
6. **Depth or Level:** The depth(or level) of a node is equal to the number of edges from tree's root node.
7. **Height:** The height of the node is the length of the path from that node to the deepest node in the tree.
8. **Subtree:** A tree that consists of a node and all its descendants.



Binary Tree:

A Binary Tree Data Structure is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. It is commonly used in computer science for efficient storage and retrieval of data, with various operations such as insertion, deletion, and traversal.

Each node can have at most two children: A binary tree is a tree data structure in which each node has at most two child nodes. This means that a node can have either zero, one, or two children.

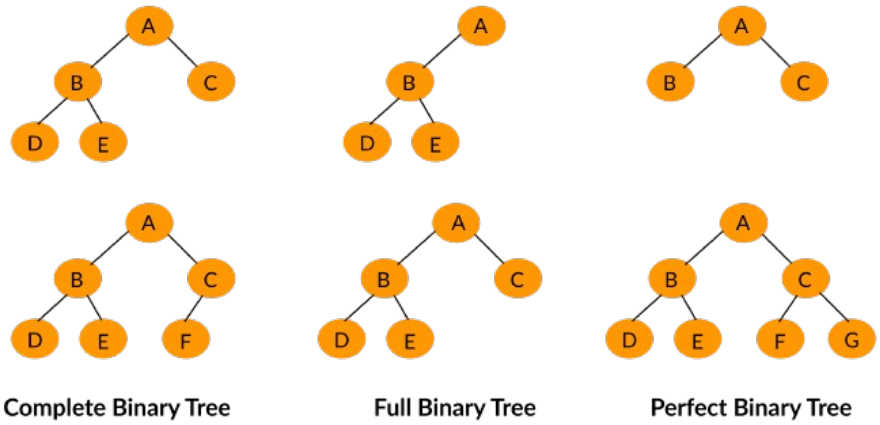
Binary trees can be implemented in many different ways, including arrays and linked lists. One common way to implement a binary tree is to use a linked list, where each node in the tree points to its left and right child nodes.

Types of Binary Tree

There are several types of binary trees, some of the common types are:

1. **Complete Binary Tree:** A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible.
2. **Full Binary Tree:** A full binary tree is a binary tree in which every node has either 0 or 2 children. In other words, every node in the tree has either two child nodes or no child nodes.

3. **Perfect Binary Tree:** A perfect binary tree is a type of binary tree in which all the leaf nodes are at the same level, and all the internal nodes have exactly two child nodes. This means that each level of the tree is completely filled, and the total number of nodes in the tree is a power of two.



Binary Search Tree:

Definition: A Binary Search Tree is a binary tree that is either empty or in which every node contains a key and satisfies the following conditions.

1. The key in the left child of a node (if it exists) is less than the key in its parent node.
2. The key in the right child of a node is greater than the key in its parent node.
3. The left and right sub trees of the root are again binary search trees.

No two entries in a binary search tree may have equal keys.

Properties of Binary Search Tree

Here are some of the important properties of BSTs:

1. **Unique values:** A BST can only have unique values in each node. This is because if there were two nodes with the same value, it would violate the binary search property.
2. **Sorted order:** The values in a BST are stored in a sorted order, making it easy to search for specific values. In-order traversal of the tree will visit the nodes in ascending order.

3. **Balanced tree:** A balanced tree is one in which the height of the left and right subtrees of every node differs by at most one. A balanced BST ensures that search, insertion, and deletion operations are performed in O(log n) time complexity on average.
4. **Maximum and minimum values:** The maximum value in a BST is the rightmost node, and the minimum value is the leftmost node.
5. **Recursive structure:** A BST is a recursive data structure, in which each subtree is itself a BST.

Advantages of Binary Search Tree

Binary search trees (BSTs) offer several advantages over other data structures. Here are some of the main advantages of BSTs:

1. **Efficient searching:** BSTs allow for efficient searching of data. Due to the sorted order of the tree, search operations can be performed in O(log n) time complexity on average. This makes BSTs an excellent choice for applications that require frequent searching, such as database indexing and retrieval.
2. **Efficient insertion and deletion:** BSTs also allow for efficient insertion and deletion of data. When adding or removing a node from a BST, the tree is automatically rebalanced to maintain the binary search property. This ensures that the tree remains efficient even after many insertions and deletions.
3. **Memory efficiency:** BSTs can be implemented using pointers or references, which allows for efficient use of memory. Compared to other data structures, such as arrays or linked lists, BSTs require less memory to store the same amount of data.
4. **Sorted order:** The sorted order of a BST makes it easy to perform operations that require the data to be sorted, such as finding the maximum or minimum value, or performing in-order traversal.
5. **Flexibility:** BSTs are highly flexible and can be adapted to suit a wide range of applications. For example, a BST can be used to implement a priority queue, a symbol table, or a balanced binary tree.

In summary, BSTs offer several advantages over other data structures, including efficient searching, efficient insertion and deletion, memory efficiency, sorted order, and flexibility. These properties make BSTs a popular choice for a wide range of applications in computer science and beyond.

Binary Search Tree

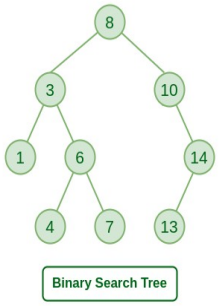
A tree is a kind of data structure that is used to represent the data in hierarchical form. It is a non-linear data structure as the data in a tree is not stored linearly or sequentially.It can be defined as a collection of objects or entities called as nodes that are linked together to simulate a hierarchy.

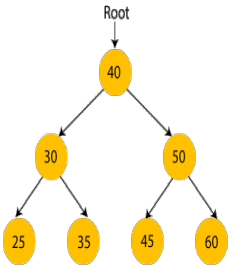
A Binary Search Tree (BST) is a special type of binary tree in which the left child of a node has a value less than the node's value and the right child has a value greater than the node's value. This property is called the BST property and it makes it possible to efficiently search, insert, and delete elements in the tree.

The root of a BST is the node that has the smallest value in the left subtree and the largest value in the right subtree. Each left subtree is a BST with nodes that have smaller values than the root and each right subtree is a BST with nodes that have larger values than the root.

Binary Search Tree is a node-based binary tree data structure that has the following properties:

- The left subtree of a node contains only nodes with keys lesser than or equal the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- This means everything to the left of the root is less than the value of the root and everything to the right of the root is greater than the value of the root. Due to this performing, a binary search is very easy.
- The left and right subtree each must also be a binary search tree.
  There must be no duplicate nodes(BST may have duplicate values with different handling approaches)



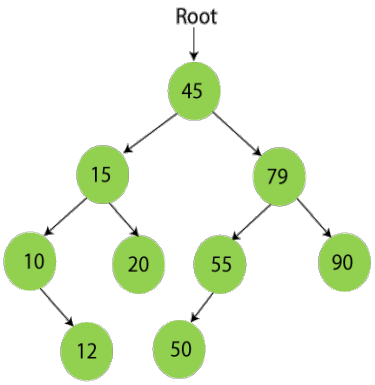



```
// Given Node node
struct node {
int key;
struct node *left, *right;
};

// Function to create a new BST node
struct node* newNode(int item)
{
struct node* temp= (struct node*)malloc(sizeof(struct node));
temp->key = item;
```

```
temp->left = temp->right = NULL;
return temp;
}
```

Ex: **45, 15, 79, 90, 10, 55, 12, 20, 50**



Searching in Binary search tree

Searching means to find or locate a specific element or node in a data structure. In Binary search tree, searching a node is easy because elements in BST are stored in a specific order. The steps of searching a node in Binary Search tree are listed as follows -

1. First, compare the element to be searched with the root element of the tree.
2. If root is matched with the target element, then return the node's location.
3. If it is not matched, then check whether the item is less than the root element, if it is smaller than the root element, then move to the left subtree.
4. If it is larger than the root element, then move to the right subtree.
5. Repeat the above procedure recursively until the match is found.
6. If the element is not found or not present in the tree, then return NULL.

```
1. START
2. Check whether the tree is empty or not
3. If the tree is empty, search is not possible
4. Otherwise, first search the root of the tree.
5. If the key does not match with the value in the root, search its subtrees.
6. If the value of the key is less than the root value, search the left subtree
7. If the value of the key is greater than the root value, search the right
subtree.
8. If the key is not found in the tree, return unsuccessful search.
9. END
```

Search Algorithm:

Search(root,item)

1. Step1-if(item=root→data)or(root=NULL)
2. return root
3. else if (item < root → data)
4. return Search(root → left, item)
5. else

6. return Search(root → right, item)
7. END if
8. Step 2 - END

Binary Search Tree Operations

Commonly performed operations on binary search tree are-



- Search Operation
- Insertion Operation
- Deletion Operation

1. Search Operation-

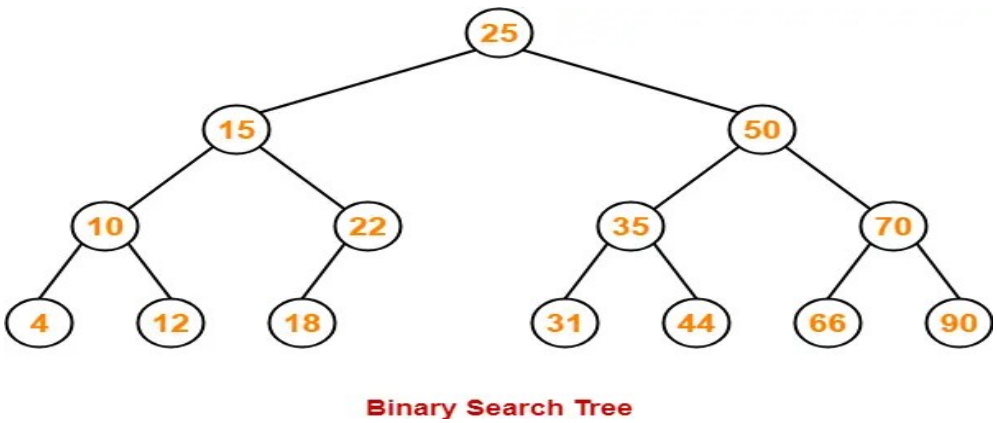
Search Operation is performed to search a particular element in the Binary Search Tree.

Rules-

For searching a given key in the BST,

- Compare the key with the value of root node.
- If the key is present at the root node, then return the root node.
- If the key is greater than the root node value, then recur for the root node's right subtree.
- If the key is smaller than the root node value, then recur for the root node's left subtree.

Example-Consider key = 45 has to be searched in the given BST-



Binary Search Tree

We start our search from the root node 25.
As 45 > 25, so we search in 25's right subtree.
As 45 < 50, so we search in 50's left subtree.
As 45 > 35, so we search in 35's right subtree.
As 45 > 44, so we search in 44's right subtree but 44 has no subtrees.
So, we conclude that 45 is not present in the above BST.

2. Insertion Operation-

Insertion Operation is performed to insert an element in the Binary Search Tree.

Rules-

The insertion of a new key always takes place as the child of some leaf node.

For finding out the suitable leaf node,

9. Search the key to be inserted from the root node till some leaf node is reached.
10. Once a leaf node is reached, insert the key as child of that leaf node.

Algorithm

1. Create a new BST node and assign values to it.

2. insert(node, key)

i) If root == NULL,

return the new node to the calling function.

ii) if root=>data < key

call the insert function with root=>right and assign the return value in root=>right.

root->right = insert(root=>right,key)

iii) if root=>data > key

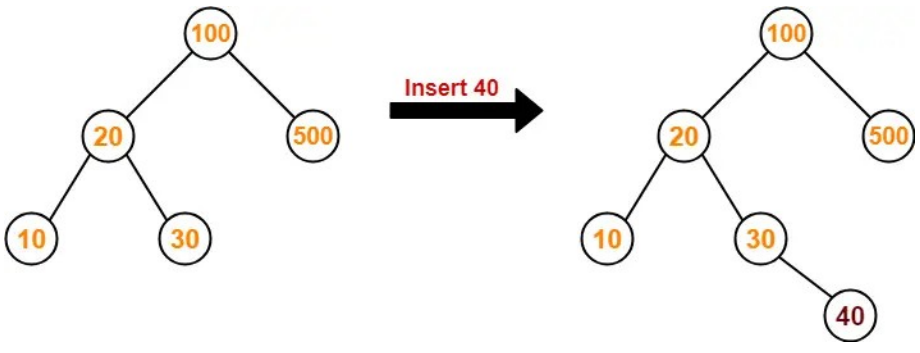
call the insert function with root->left and assign the return value in root=>left.

root=>left = insert(root=>left,key)

3. Finally, return the original root pointer to the calling function.

Example-

Consider the following example where key = 40 is inserted in the given BST-



We start searching for value 40 from the root node 100.
As 40 < 100, so we search in 100's left subtree.
As 40 > 20, so we search in 20's right subtree.
As 40 > 30, so we add 40 to 30's right subtree.

Insertion Program

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int key;
    struct node *left;
    struct node *right;
};

//this function will return the new node with the given value
struct node *getNewNode(int val)
{
    struct node *newNode = malloc(sizeof(struct node));
    newNode->key   = val;
    newNode->left  = NULL;
    newNode->right = NULL;

    return newNode;
}

struct node *insert(struct node *root, int val)
{
    /*
     * It will handle two cases,
     * 1. if the tree is empty, return new node in root
     * 2. if the tree traversal reaches NULL, it will return the new node
     */
    if(root == NULL)
        return getNewNode(val);
    /*
     * if given val is greater than root->key,
     * we should find the correct place in right subtree and insert the new node
     */
    if(root->key < val)
        root->right = insert(root->right,val);
    /*
     * if given val is smallar than root->key,
     * we should find the correct place in left subtree and insert the new node
     */
```

```
    else if(root->key > val)
        root->left = insert(root->left,val);
    /*
     * It will handle two cases
     * (Prevent the duplicate nodes in the tree)
     * 1.if root->key == val it will straight away return the address of the
root node
     * 2.After the insertion, it will return the original unchanged root's
address
     */
    return root;
}

/*
 * it will print the tree in ascending order
 * we will discuss about it in the upcoming tutorials
 */
void inorder(struct node *root)
{
    if(root == NULL)
        return;
    inorder(root->left);
    printf("%d ",root->key);
    inorder(root->right);
}

int main()
{
    struct node *root = NULL;
    root = insert(root,100);
    root = insert(root,50);
    root = insert(root,150);
    root = insert(root,50);

    inorder(root);

    return 0;
}
```

3. Deletion Operation-

Deletion Operation is performed to delete a particular element from the Binary Search Tree.

When it comes to deleting a node from the binary search tree, following three cases are possible-

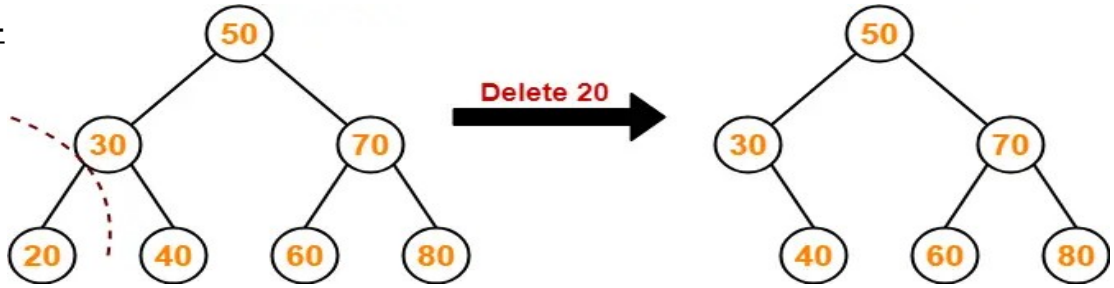
Case-01: Deletion Of A Node Having No Child (Leaf Node)-

Just remove / disconnect the leaf node that is to deleted from the tree.

Example-

Consider the following example where node with value = 20 is deleted from the BST-

Case-02:



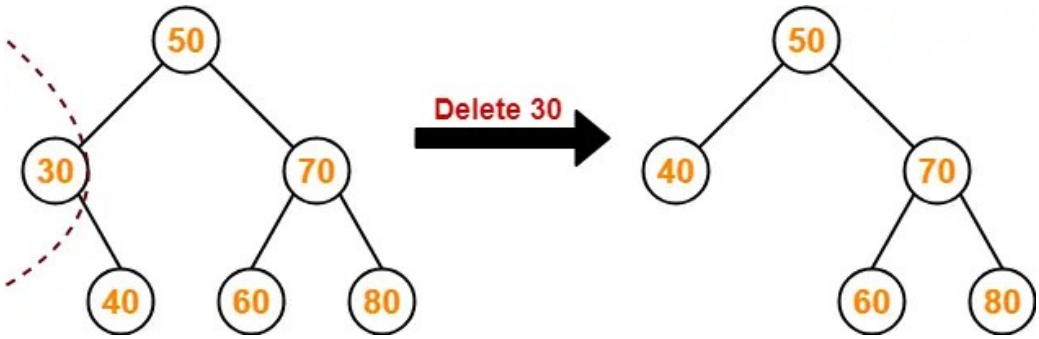
Deletion Of A Node Having Only One Child-

Just make the child of the deleting node, the child of its grandparent.

Example-

Consider the following example where node with value = 30 is deleted from the BST-

Case-03:



Deletion Of A Node Having Two Children-

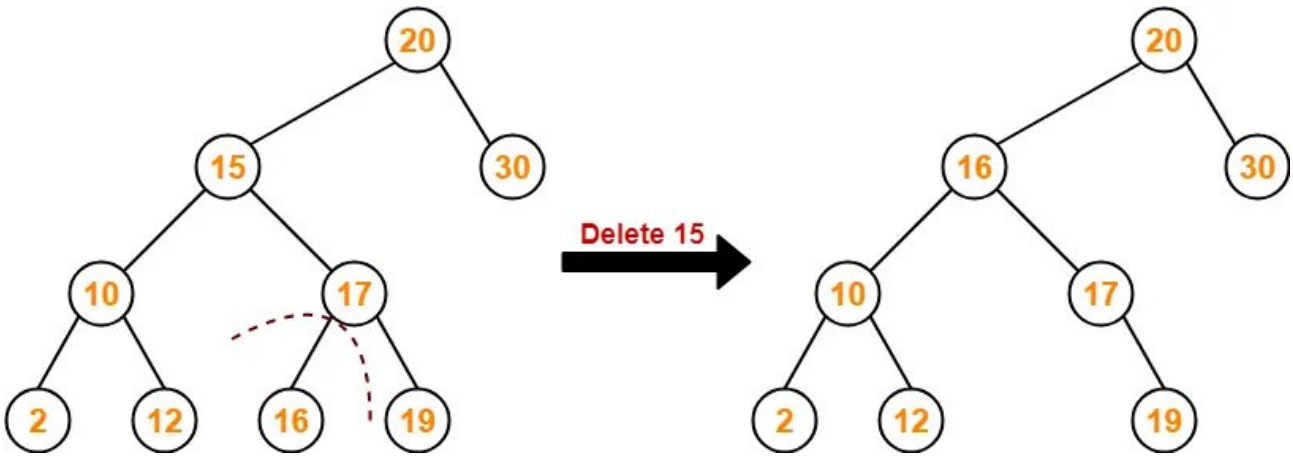
A node with two children may be deleted from the BST in the following two ways-

Method-01:

- Visit to the right subtree of the deleting node.
- Pluck the least value element called as inorder successor.
- Replace the deleting element with its inorder successor.

Example-

Consider the following example where node with value = 15 is deleted from the BST-

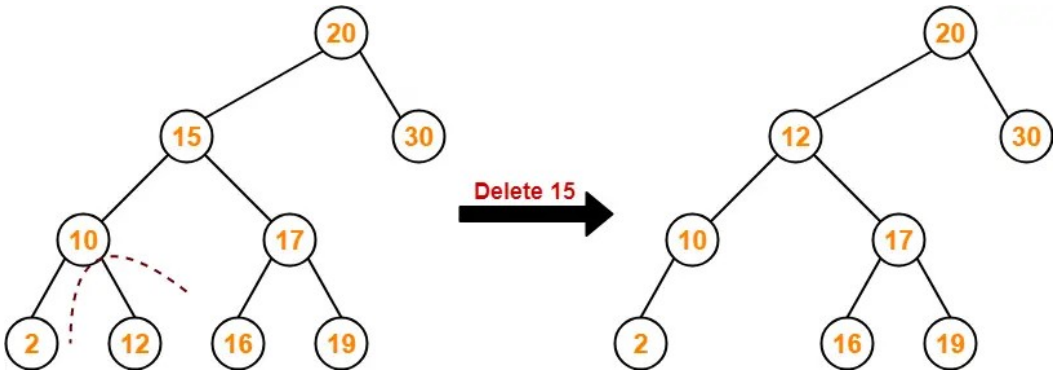


Method-02:

- Visit to the left subtree of the deleting node.
- Pluck the greatest value element called as inorder predecessor.
- Replace the deleting element with its inorder predecessor.

Example-

Consider the following example where node with value = 15 is deleted from the BST-




```
// C function to search a given key in a given BST

#include <stdio.h>
#include <stdlib.h>

struct node {
    int key;
    struct node *left, *right;
};

// A utility function to create a new BST node
struct node* newNode(int item)
{
    struct node* temp
        = (struct node*)malloc(sizeof(struct node));
    temp->key = item;
```

```
    temp->left = temp->right = NULL;
    return temp;
}

// A utility function to insert
// a new node with given key in BST
struct node* insert(struct node* node, int key)
{
    // If the tree is empty, return a new node
    if (node == NULL)
        return newNode(key);

    // Otherwise, recur down the tree
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    // Return the (unchanged) node pointer
    return node;
}

// Utility function to search a key in a BST
struct node* search(struct node* root, int key)
{
    // Base Cases: root is null or key is present at root
    if (root == NULL || root->key == key)
        return root;

    // Key is greater than root's key
    if (root->key < key)
        return search(root->right, key);

    // Key is smaller than root's key
    return search(root->left, key);
}

// Driver Code
int main()
{
    struct node* root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
    insert(root, 40);
    insert(root, 70);
    insert(root, 60);
    insert(root, 80);

    // Key to be found
    int key = 6;

    // Searching in a BST
```

```
    if (search(root, key) == NULL)
        printf("%d not found\n", key);
    else
        printf("%d found\n", key);

    key = 60;

    // Searching in a BST
    if (search(root, key) == NULL)
        printf("%d not found\n", key);
    else
        printf("%d found\n", key);
    return 0;
}
```